LAPORAN PRAKTIKUM
PEMROGRAMAN PARALEL



Praktikum 1
Pemrograman Shared Memory dengan OpenMP
Kode Asesmen A05

Disusun oleh:

Nama : Sachio Aji

NIM : 09011482528013

Fakultas Ilmu Komputer
Universitas Sriwijaya
2026

1. Tujuan

Praktikum ini bertujuan melatih cara memparalelkan program serial dengan OpenMP, memastikan hasilnya tetap benar, dan melihat bagaimana kebijakan scheduling memengaruhi waktu eksekusi pada beban kerja yang tidak seragam. Setelah praktikum, mahasiswa diharapkan mampu:

1. memparalelkan perulangan dengan #pragma omp parallel for dan menentukan cakupan variabel dengan benar
2. menggabungkan hasil dengan klausa reduction;
3. membandingkan schedule(static), schedule(dynamic, k), dan schedule(guided) secara terukur;
4. menghitung speedup dan efisiensi dari hasil pengukuran.

Ada dua program yang diuji: perkalian matriks yang bebannya seragam, dan penghitungan bilangan prima sampai 10^7 yang bebannya tidak seragam.

2. Dasar Teori Singkat

Direktif parallel for membagi iterasi sebuah perulangan ke beberapa thread, dengan syarat setiap iterasi tidak saling bergantung. Variabel yang dideklarasikan di luar region paralel bersifat shared, sehingga berisiko terkena race condition ketika beberapa thread menulis ke variabel yang sama. Klausa reduction mengatasi hal ini dengan memberi setiap thread salinan variabelnya sendiri, lalu menggabungkan semuanya di akhir region. Sementara itu, klausa schedule mengatur cara iterasi dibagikan ke thread. Pilihan yang paling cocok bergantung pada apakah biaya tiap iterasi seragam atau tidak.

3. Lingkungan dan Metode

3.1 Lingkungan

Tabel 1. Lingkungan

Prosesor	Core fisik	Core logis	Memori	Versi GCC	Opsi kompilasi
AMD Ryzen 5 5625U with Radeon Graphics	6	12 (2 thread per core)	2,8 GiB (alokasi WSL)	GCC 15.2.0 (Ubuntu 15.2.0-16ubuntu1)	-O2 -fopenmp (matmul ditambah -lm)

Percobaan dijalankan di Ubuntu pada WSL. Memori 2,8 GiB adalah alokasi WSL, bukan RAM fisik laptop.

3.2 Perkalian matriks

Perulangan terluar (i) diparalelkan dengan #pragma omp parallel for. Ukuran matriks yang dipakai adalah n = 1024, karena waktu serialnya (sekitar 5 detik) sudah melewati batas 2 detik pada modul. Hasil versi paralel dibandingkan dengan versi serial memakai toleransi relatif 10^{-9}.

3.3 Bilangan prima

Perulangan x = 2 sampai 10^7 diparalelkan dengan reduction(+:jumlah) dan schedule(runtime). Kebijakan scheduling diatur lewat variabel lingkungan OMP_SCHEDULE dengan tiga pilihan: static, dynamic,1000, dan guided. Jumlah bilangan prima yang benar adalah 664.579.

3.4 Pengukuran

Setiap konfigurasi dijalankan 3 kali dengan 1, 2, 4, dan 8 thread, dan tabel memuat median dari tiga pengulangan tersebut. Waktu diukur dengan omp_get_wtime(). Rumus yang dipakai:

- Speedup = waktu serial / waktu paralel;
- Efisiensi = speedup / jumlah thread.

Khusus program prima, speedup dihitung terhadap waktu 1 thread pada kebijakan scheduling yang sama.

4. Hasil Pengamatan

Tabel 2. Perkalian matriks, n = 1024 (median tiga pengulangan, waktu dalam detik)

Thread	Waktu serial	Waktu paralel	Speedup	Efisiensi	Benar?
1	4,835	5,009	0,965	0,965	ya
2	4,980	2,591	1,922	0,961	ya
4	5,357	1,714	3,125	0,781	ya
8	5,328	1,066	4,998	0,625	ya

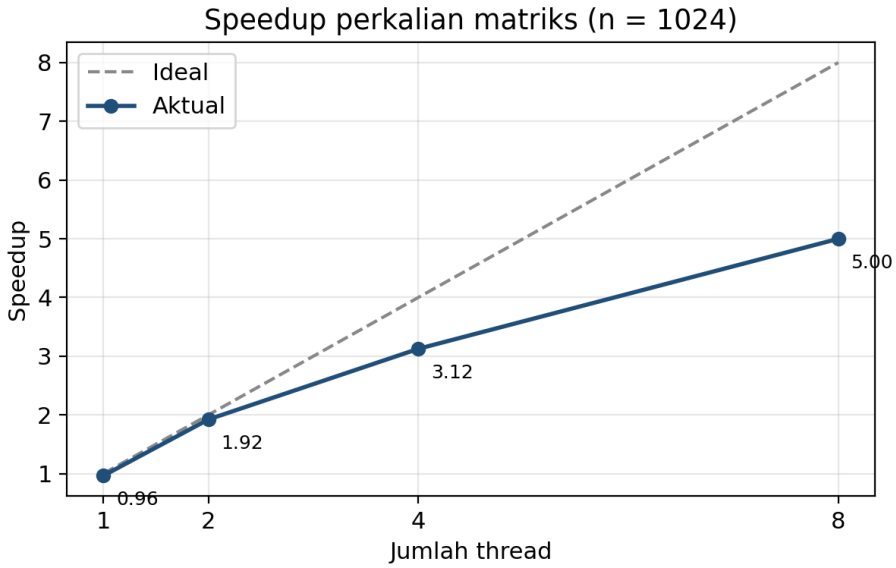
Tabel 3. Bilangan prima sampai 10^7, waktu per kebijakan scheduling (median, detik)

Kebijakan	1 thread	2 thread	4 thread	8 thread	Speedup 8 thread	Hasil = 664.579?
static	3,512	1,927	1,112	0,773	4,54	ya
dynamic, 1000	2,910	1,521	0,840	0,618	4,71	ya
guided	2,939	1,517	0,846	0,600	4,90	ya

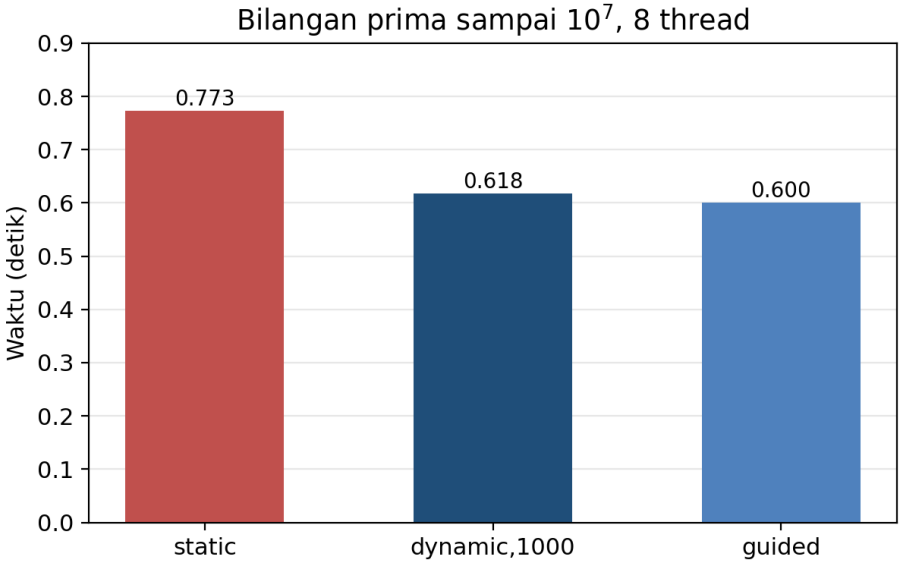
Speedup 8 thread pada Tabel 3 diperoleh dari waktu 1 thread dibagi waktu 8 thread untuk kebijakan yang sama.

Tabel 4. Percobaan tanpa reduction (4 thread, 3 kali)

Percobaan	Run 1	Run 2	Run 3
Nilai jumlah	179.718	171.361	170.777
Nilai yang benar	664.579	664.579	664.579



Grafik 1. Speedup perkalian matriks terhadap jumlah thread, dengan garis ideal



Grafik 2. Waktu 8 thread untuk tiga kebijakan scheduling

5. Analisis

5.1 Perulangan yang diparalelkan dan variabel private

Perulangan yang diparalelkan adalah perulangan terluar (i). Variabel i, j, k, dan s bersifat private: i adalah variabel perulangan yang dibagi ke thread, sedangkan j, k,

dan s dideklarasikan di dalam region paralel sehingga tiap thread punya salinan sendiri. Variabel A, B, C, dan n bersifat shared. Pilihan ini aman dari race condition karena:

- A dan B hanya dibaca, tidak pernah ditulis;
- setiap iterasi i menulis ke baris C yang berbeda, jadi tidak ada dua thread yang menulis ke elemen yang sama;
- akumulator s bersifat private untuk tiap iterasi, dan urutan penjumlahan pada k sama persis dengan versi serial.

Buktinya, kolom Benar? pada Tabel 2 bernilai ya untuk semua jumlah thread.

5.2 Speedup terhadap speedup ideal

Speedup perkalian matriks mendekati ideal pada jumlah thread kecil, tetapi makin menjauh seiring bertambahnya thread: 1,92 pada 2 thread (ideal 2), 3,13 pada 4 thread (ideal 4), dan 5,00 pada 8 thread (ideal 8), seperti terlihat pada Grafik 1. Efisiensinya juga turun dari 0,96 menjadi 0,78, lalu 0,63. Ada dua faktor yang mungkin menjadi penyebabnya:

• **Akses memori dan cache.** Matriks B diakses per kolom (B[k*n+j]), pola yang tidak ramah cache. Semua thread juga berebut bandwidth memori dan cache L3 yang sama, sehingga menambah thread tidak memberi tambahan kecepatan yang sebanding.

• **Overhead dan batas perangkat keras.** Ada biaya untuk membuat thread dan sinkronisasi di akhir region. Selain itu, prosesor ini adalah seri hemat daya (U) yang cenderung menurunkan frekuensi ketika banyak core aktif.

Pada 1 thread, speedup 0,965 (sedikit di bawah 1) menunjukkan adanya overhead OpenMP tanpa keuntungan dari paralelisme.

5.3 Pengaruh scheduling pada program prima

Biaya memeriksa satu bilangan x sebanding dengan √x, sehingga total biaya sampai x tumbuh sebanding dengan x^1,5. Pada static dengan 4 thread, iterasi dipotong menjadi 4 blok yang berurutan. Thread terakhir kebagian bilangan 7,5 sampai 10 juta dan menanggung sekitar 1 − 0,75^1,5 ≈ 35% dari total kerja, bukan 25%. Thread lainnya selesai lebih cepat lalu menganggur (load imbalance), sehingga

waktu total ditentukan oleh thread terakhir. Perkiraan perlambatannya sekitar 0,35/0,25 = 1,4×. Data mendukung perkiraan ini: pada 4 thread, static butuh 1,112 s sedangkan dynamic 0,840 s (rasio 1,32×).

Sebaliknya, dynamic,1000 dan guided membagikan pekerjaan secara adaptif: thread yang lebih dulu selesai langsung mengambil potongan berikutnya, sehingga bebannya merata. Pada 8 thread (Grafik 2), waktu guided (0,600 s, speedup 4,90) sedikit lebih baik daripada dynamic,1000 (0,618 s, speedup 4,71), dan keduanya jauh lebih baik daripada static (0,773 s, speedup 4,54). Selisih guided dan dynamic masih berada dalam rentang variasi antar-ulangan (guided 0,580 sampai 0,643 s; dynamic 0,593 sampai 0,630 s), jadi keduanya bisa dianggap setara. Kesimpulannya, guided memang terbaik secara angka, tetapi tidak berbeda nyata dari dynamic,1000, dan keduanya jelas mengungguli static.

Catatan: pada 1 thread, static (3,512 s) tampak lebih lambat daripada dynamic dan guided (sekitar 2,9 s). Padahal dengan 1 thread, kebijakan scheduling seharusnya tidak berpengaruh. Selisih ini kemungkinan berasal dari variasi kondisi sistem (WSL, frekuensi CPU), bukan dari kebijakannya. Karena speedup static dihitung dari waktu 1 thread tersebut, nilainya bisa sedikit tergelembung.

5.4 Percobaan tanpa reduction

Tanpa reduction, tiga run menghasilkan 179.718; 171.361; dan 170.777 (Tabel 4). Hasilnya berbeda di setiap run dan jauh di bawah nilai benar 664.579 (hilang sekitar 74%). Penyebabnya, jumlah bersifat shared, sedangkan operasi jumlah += ... terdiri dari tiga langkah (baca, ubah, tulis) yang tidak atomik. Ketika beberapa thread membaca nilai lama secara bersamaan, update dari satu thread menimpa update thread lain, sehingga banyak penambahan hilang. Waktu eksekusi tetap sekitar 1,1 detik dan program tidak crash, jadi race condition ini tidak kelihatan kecuali hasilnya diperiksa. Dengan reduction(+:jumlah), setiap thread punya salinan lokal yang digabung di akhir, dan hasilnya selalu 664.579. Klausa reduction sudah dikembalikan setelah percobaan.

5.5 Dampak jumlah core fisik kurang dari 8

Mesin ini memiliki 6 core fisik (12 thread logis), sehingga 8 thread melebihi jumlah core fisik. Sebagian thread terpaksa berbagi core lewat SMT, sehingga berbagi unit eksekusi dan cache. Dampaknya terlihat pada Tabel 2: dari 4 ke 8 thread, speedup hanya naik dari 3,13 menjadi 5,00 (1,6× untuk 2× thread) dan efisiensi turun dari 0,78 menjadi 0,63.

Tambahan kecepatan dari SMT jauh lebih kecil daripada dari core fisik tambahan. Pada Tabel 3 hal serupa terjadi: speedup 8 thread hanya 4,5 sampai 4,9, bukan mendekati 8.

6. Keterbatasan

- Pengukuran dilakukan di laptop dengan WSL, sehingga ada variasi antar-run akibat frekuensi CPU, suhu, dan proses latar belakang. Waktu serial yang sama berkisar 4,7 sampai 6,5 s antar run, jadi ketidakpastiannya sekitar ±10%. Median dipakai untuk meredam variasi ini.
- Setiap konfigurasi hanya diulang 3 kali, sehingga selisih kecil (seperti antara guided dan dynamic) tidak bisa disimpulkan secara tegas.

7. Kesimpulan

1. Memparalelkan perulangan i pada perkalian matriks, dengan variabel perantara bersifat private, menghasilkan hasil yang benar dan speedup 4,998 pada 8 thread (efisiensi 0,625).
2. Speedup berada di bawah ideal karena keterbatasan memori dan cache, overhead OpenMP, serta 8 thread melebihi 6 core fisik.
3. Pada beban tidak seragam (prima), static paling lambat karena load imbalance. Kebijakan dynamic,1000 dan guided setara dan jauh lebih baik, dengan guided sedikit unggul (0,600 s pada 8 thread).
4. Klausa reduction wajib dipakai untuk akumulasi variabel shared. Tanpa reduction, hasilnya salah dan tidak konsisten (sekitar 171 sampai 180 ribu dari seharusnya 664.579).

Lampiran

- Kode: src/matmul.c, src/prima.c, src/prima_b4_tanpa_reduction.c
- Data mentah: hasil/matmul.txt, hasil/prima.txt
- Repositori: [isi tautan repositori]